

Reservoir Computing with the Human Connectome Project

Tebe Nigrelli

Giusi Scozzi

Antonio Perazza

Kevin Cesenj

Vecdi Gokmen

Emilio Vitali

Christina E. Christodoulou

Lorena Pavel

Angelo Spahiu

Following the kind direction of Postdoc Researcher Victor Buendia

1. Introduction

The brain processes information through networks of neurons connected by synapses. Reservoir computing is a machine learning approach inspired by this idea: a fixed recurrent network, called a **reservoir**, transforms an input signal into a rich set of internal states, from which a simple linear readout is trained to make predictions. Because only the readout is trained, the method is computationally efficient while still capable of modeling complex and chaotic signals.

In this work, we use structural connectivity data from the Human Connectome Project, which provides a detailed map of connections between brain regions. This allows us to replace the usual random reservoir with a biologically grounded one, directly embedding real brain structure into the model.

The reservoir is driven by a chaotic input signal and trained to predict its next time step. We then progressively remove nodes and connections to simulate structural damage, measuring how prediction performance degrades. This approach allows us to investigate how the topology of the connectome supports computation and how robust it is to perturbations.

2. Reservoir dynamics

A useful way to think about the reservoir is through the analogy of a lake. When rain falls on its surface, each drop creates ripples that spread, interact, and persist for some time before fading. By observing the surface you can infer something about the rain that caused it, without seeing the individual drops. The reservoir works in the same way: an input signal perturbs the network, and this perturbation propagates through the connections and influences future states.

This intuition can be formalized by describing the reservoir as a dynamical system. The state of

neuron i at time t , denoted by $r_{i(t)}$, evolves according to

$$\frac{dr_i}{dt} = -r_i(t) + S\left(\sum_{j=1}^N W_{ij}r_j(t)\right)$$

The term $-r_{i(t)}$ represents a decay, while the second term captures how the neuron is influenced by other neurons through the weight matrix W , after applying a nonlinear transformation. The current state of the reservoir can therefore be seen as a compressed representation of recent inputs.

In practice, its dynamics are approximated by evaluating the state at discrete time steps. The time derivative is expressed as

$$\frac{dr_i(t)}{dt} = \frac{r_i(t + \delta t) - r_i(t)}{\delta t}$$

which leads, via Euler discretization, to the update rule

$$r_i(t + \delta t) = r_i(t) + \delta t \frac{dr_i}{dt}$$

The richness of the reservoir dynamics emerges directly from its structure: the irregular and highly interconnected network transforms even simple inputs into complex patterns of activity. Different neurons respond in different ways, creating a diverse set of signals within the network. This diversity is useful because it provides a wide range of features that can later be used to extract meaningful information from the system.

This balance is governed by the properties of the recurrent matrix W , which determine whether activity decays, persists, or grows over time. As a result, the reservoir can operate in different dynamical regimes, which play a key role in its computational capabilities.

To study this behavior in a controlled setting, we first consider a Gaussian random reservoir, where the recurrent matrix W is constructed from

independent Gaussian entries. In this case, the weights are sampled as

$$W_{ij} \sim \mathcal{N}\left(0, \frac{J^2}{N}\right)$$

where N is the number of neurons and J controls the overall strength of recurrent interactions.

The stability of the system can be understood by analyzing the eigenvalues of W . For large N , Girko's circular law states that these eigenvalues are approximately distributed within a disc of radius J in the complex plane. The spectral radius, defined as the magnitude of the largest eigenvalue, therefore determines whether perturbations decay or grow over time. In discrete time, stability requires eigenvalues to lie within the unit circle, while in continuous time it depends on the real part of the eigenvalues.

As a result, different regimes can be identified: for $J < 1$, activity decays and the system is stable; for $J \approx 1$, the system operates near a critical regime; and for $J > 1$, the dynamics become unstable or chaotic.

External input enters the system through the input weight matrix W^{in} . In the state equation it appears as $\sum_j W_{ij}^{\text{in}} h_{j(t)}$ and determines how the input signal affects each neuron before the nonlinearity is applied. The matrix W^{in} is fixed and randomly initialized, typically with Gaussian entries. It determines how different components of the input signal are distributed across the neurons of the reservoir. As a result, the state of the reservoir reflects both its internal dynamics and the incoming input.

The goal is to use the reservoir state to predict the next value of the input signal, formally expressed as $h(t) \approx h(t + \delta t)$. To do this, the model maps the reservoir state $r(t)$ to an output through a linear transformation: $h(t) = W_{\text{readout}} * r(t)$ where W_{readout} contains the readout weights.

Unlike the internal and input weights, which remain fixed, the readout weights are the only parameters that are adjusted during training. The objective is to approximate the next-step value of the input signal, so that $h(t) \approx h(t + \delta t)$.

Finally, it is important to note that the internal structure of the reservoir is not trained. The dynamics arise naturally from the fixed connections and the interaction between neurons. This means that the computational power of the

system comes directly from these internal dynamics, rather than from adjusting them during learning.

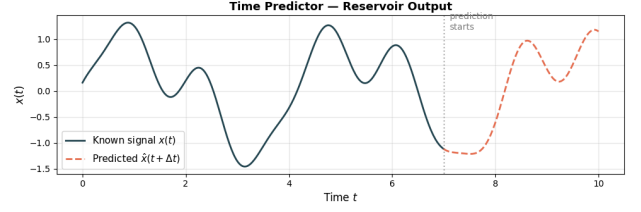


Figure 1: Example of time-series prediction using the reservoir.

3. Dynamics overview

Building on the framework introduced above, we now investigate how the reservoir dynamics change as the coupling parameter J is varied.

Figure 2 shows representative neuron trajectories for different values of J . For $J < 1$, all trajectories quickly converge to zero, indicating that the system settles into a stable fixed point and loses any dependence on its initial state. As J approaches the critical value, the decay of activity becomes slower and trajectories remain active for longer times, reflecting an increased ability to retain past information. For $J > 1$, the dynamics no longer converge and instead become irregular and highly variable, indicating a transition to a chaotic regime.

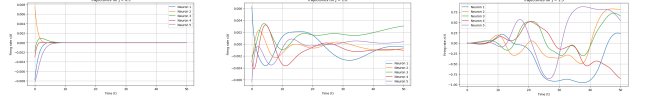


Figure 2: Neuron activations over time for different values of J . From left to right: $J < 1$, $J \approx 1$, and $J > 1$.

To quantitatively characterize this transition, we measure the temporal variability of neural activity. Specifically, we simulate networks of $N = 400$ neurons over $T = 50$ time units with time step $dt = 0.1$, and compute the variance of each neuron's activity over time. This quantity is then averaged across neurons and over 10 independent realizations for each value of J , using fixed random seeds to ensure reproducibility.

The results are reported in Fig. 3, where the mean temporal variance is plotted as a function of J . For small values of J , the variance remains close to zero, confirming that all trajectories converge rapidly. As J increases beyond the critical regime, the variance grows sharply, reflecting the emergence of sustained and fluctuating activity.

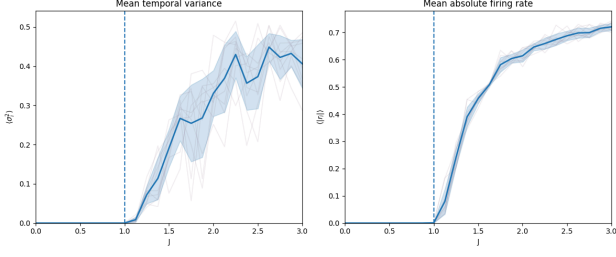


Figure 3: Mean temporal variance as a function of J , averaged over 10 trials (seed = 42).

These dynamical regimes have direct implications for the memory properties of the reservoir. In the stable regime ($J < 1$), the system quickly forgets past inputs, as all trajectories collapse to a single fixed point. Near the critical regime ($J \approx 1$), slower decay allows the reservoir to retain information over longer times, enhancing its memory capacity. In contrast, for $J > 1$, the dynamics become highly sensitive to perturbations, leading to a breakdown of consistent behavior and a violation of the Echo State Property.

These effects are also visible at the population level. As shown in Fig. 4, the distribution of neuron activations broadens over time as J increases, transitioning from a narrow concentration around zero to a wide and heterogeneous spread, further illustrating the shift from stable to chaotic dynamics.

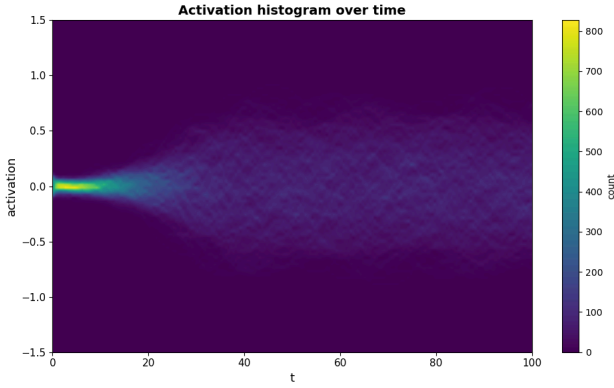


Figure 4: Activation histogram over time, using $N=500$, $T=100$, step size=0.1 (seed = 42).

4. Training Process

Before diving into the experimental results, it is helpful to outline our overall implementation strategy. We structured the project into two distinct phases. First, we built and validated our entire predictive pipeline using standard, randomly generated Gaussian networks. This baseline ensured our code could successfully predict the chaotic Lorenz attractor using a controlled, well-understood foundation. Once the pipeline proved stable, we swapped the random matrices for the

empirical human connectome dataset to evaluate how a real brain topology handles the same task.

In Reservoir Computing, the internal network is fixed; therefore, our “training” pipeline is broken into two practical, data-driven stages: simulating the reservoir to harvest internal states, and tuning the regularization to optimize the readout weights.

To generate the data required for the readout optimization, we simulate the reservoir’s response to the target signal. This is done using **Teacher Forcing**, a technique where the reservoir is driven exclusively by the true ground-truth Lorenz signal. As the data flows in, we record the activation values of all neurons at every time step into a state matrix R .

During this phase, we apply a washout period of 200 time steps. By discarding these initial states, we eliminate the transient effect of the network’s zero-state initialization, ensuring the collected matrix R captures only the synchronised, steady-state chaotic dynamics of the Lorenz attractor.

To visualize this process, Figure 5 illustrates the post-washout input signal and the corresponding internal activations of a random sample of 8 neurons. For this baseline Gaussian reservoir, we utilized $N = 250$ neurons, a spectral radius $J = 0.8$, and an input scaling factor of 0.1. The plot confirms that the network exhibits heterogeneous dynamics without saturating the activation functions, providing a high-quality dimensional expansion for the regression model.

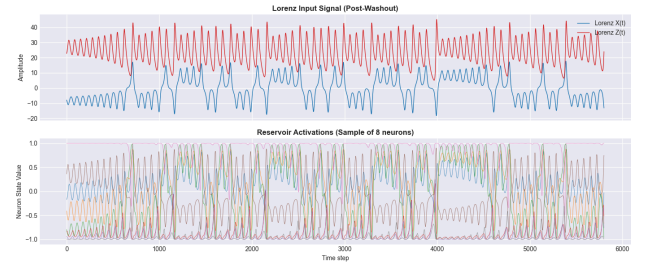


Figure 5: Reservoir Activations (seed = 42).

Once the state matrix R is collected, the output weights W_{readout} are computed using ridge regression. To learn the forward dynamics of the system, a temporal shift is applied: the reservoir states at time t are used to predict the target signal at time $t + 1$. The corresponding target values are stored in the matrix H_{target} .

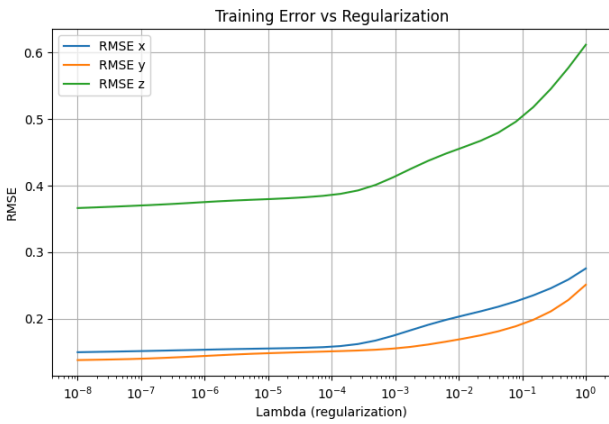
The weights are then obtained by solving a regularized least-squares problem:

$$W_{\text{readout}} = \arg \min_W |WR - H_{\text{target}}|^2 + \lambda |W|^2$$

where λ is a regularization parameter that penalizes large weights, helping prevent overfitting, meaning the model does not fit noise or small fluctuations in the training data, which would otherwise harm its ability to generalize to new inputs.

Additionally, we augment the state matrix with a constant bias term to account for spatial offsets without distorting the weights.

Our primary experimental focus during this phase was the empirical tuning of the regularization parameter λ . To understand how this parameter affects our model, we ran a sweep on our baseline network ($N = 250$), testing 30 different λ values ranging on a logarithmic scale from $10^{\{-18\}}$ to 1.



As the plot shows, increasing the regularization penalty steadily increases the Root Mean Square Error (RMSE) on the training set across all three spatial axes. The model fits the training data best when λ is extremely small (around $10^{\{-8\}}$).

5. Human Connectome Data and Network Analysis

We now turn to the human connectome data, investigating how its structure can be represented as a graph and later used as a biologically grounded reservoir.

The dataset comes from a preprocessed version of the Human Connectome Project, where each subject is stored as a .graphml file. In this format, each file represents the structural connectivity of the brain.

Each file contains information about brain regions and the connections between them. The nodes correspond to individual brain regions, while the edges represent structural connections, such as white matter fiber tracts.

From a graph theory perspective, this structure can be modeled as a graph $G = (V, E)$, where V represents the set of nodes and E represents the set of edges. The graph is undirected, so the connectivity matrix is symmetric, meaning $W_{ij} = W_{ji}$.

For the graph we analyzed, there are 463 nodes. This choice is not arbitrary, but comes from using a specific brain parcellation that balances anatomical detail with computational cost. Using this resolution allows us to work with a network that is still manageable in simulations, while preserving enough structure to capture meaningful connectivity patterns.

In addition, having a relatively large number of nodes is important because it makes the statistical properties of the network more clearly visible. In smaller graphs, these properties can be dominated by fluctuations due to finite size effects, meaning that random variations can hide the underlying structure.

Each node carries anatomical information such as its position in 3D space, its hemisphere (left or right), and its region label. The edges describe how regions are connected and include features like the mean fiber length, the fractional anisotropy (FA), and the number of fibers.

To make the data easier to work with, we separated the graph into node-level and edge-level information. This allows us to analyze both the spatial organization of the regions and the structure of the connections.

At the node level, the spatial positions of the regions show a clear 3D organization of the brain. The nodes are distributed in a roughly symmetric way, with a visible separation between the two hemispheres. Most regions form a compact structure, while a few nodes appear more spread out at the boundaries. This confirms that the graph preserves the anatomical layout of the brain.

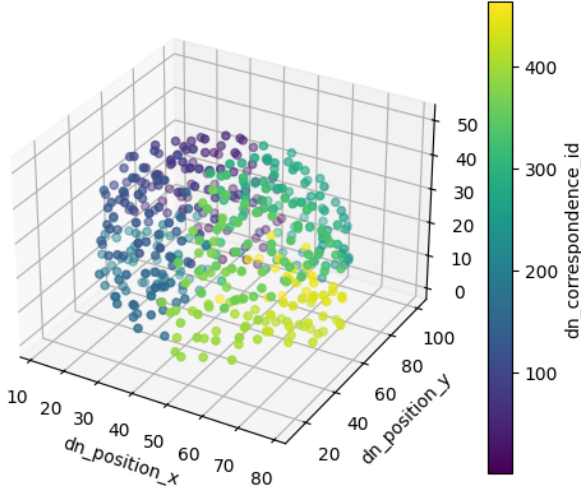


Figure 7: 3D visualization of the connectome nodes, colored by hemisphere (seed = 42).

At the edge level, the connection features are clearly not evenly distributed. In particular, the number of fibers ranges from about 1 to over 4600, with a median around 22, meaning that most connections are relatively weak. The distribution shows a strong right-skewed shape, with a peak at low values and a long tail, indicating that only a small number of edges have much larger weights than the rest.

This behavior is supported by the high skewness (≈ 6.37) and very large kurtosis (≈ 63), which indicate a heavy-tailed distribution with extreme values.

For the fiber length, the values are spread over a wider range (roughly between 10 and 100) and show a smoother decay, with moderate skewness.

The FA distribution (the middle histogram) is more concentrated and decreases rapidly as values increase. Visually, this resembles an exponential-like decay, where most values are near the lower end and larger values become increasingly rare. This is only a qualitative observation based on the histogram, and a formal statistical test would be needed to confirm it.

Overall, this confirms that the connectome is not a uniform network, but a heterogeneous one, where some connections play a more important role than others.

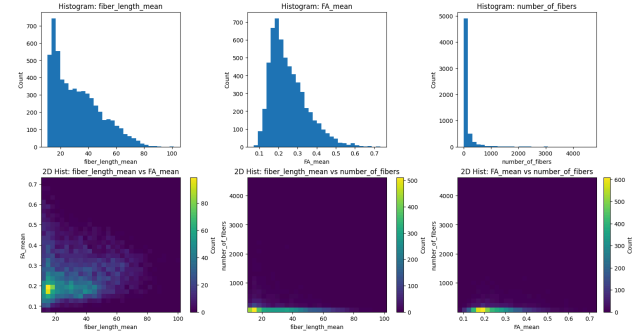


Figure 8: Histograms and pairwise 2D histograms of the main edge features (seed = 42).

We further analyze how these edge features relate to each other through correlation analysis. Overall, most relationships are weak. The most notable pattern is a moderate negative correlation between fiber length and number of fibers ($\rho \approx -0.37$), indicating that longer connections tend to involve fewer fibers.

In contrast, correlations involving FA are much weaker ($\rho \approx 0.08$ with fiber length and $\rho \approx 0.04$ with number of fibers), suggesting that FA captures a different aspect of connectivity.

To compute these correlations, we use Spearman's rank correlation instead of Pearson's correlation. This choice is motivated by the strongly skewed distributions of the edge features, particularly the number of fibers, as observed in the histograms. In such cases, Pearson correlation can be dominated by a few extreme values, whereas Spearman correlation, based on rank ordering, provides a more robust measure of association.

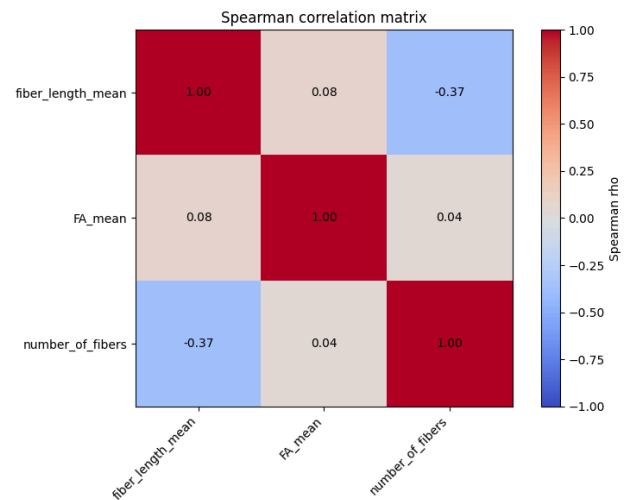


Figure 9: Spearman correlation matrix for the main structural edge features (seed = 42).

Finally, we observe that the network has a non-trivial structure, with groups of nodes that are more strongly connected among themselves. This kind of organization is referred to as **community**

structure and shows that the connectome is not a random graph, but a structured one.

5.1. Training and prediction on Lorentz System

In this section we introduce the chaotic system we will be using to test our reservoir, chosen due to its simplicity and rich nonlinear dynamics.

The Lorenz system is a set of 3 coupled ordinary differential equations, developed by Edward Lorenz while studying atmospheric convection. It is a famous example of deterministic chaos, as the equations have no random elements; however, they are extremely sensitive to initial conditions and, in some cases, display no periodic behavior.

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

The parameters ρ , σ and β are constants to be chosen, while $x(t)$, $y(t)$ and $z(t)$ are the time-dependent state variables.

The sensitivity to initial conditions makes the system impossible to predict over an infinite period of time. While the system is fully deterministic, meaning it is in theory possible to know the state of the system far into the future, even small rounding errors in our numerical integration methods get amplified exponentially over time, making long-term predictions infeasible.

Another important property of the Lorenz system is that, although its trajectories are chaotic and highly sensitive to initial conditions, they do not grow without bound or diverge to infinity. Instead, the solutions remain confined to a bounded region of phase space, forming the famous butterfly-shaped Lorenz attractor.

This property makes the system especially useful for reservoir computing, as besides testing short-term prediction accuracy, it also allows us to evaluate whether the reservoir has learned the overall qualitative dynamics of the system. A good model should reproduce the same bounded attractor structure even after exact predictions have diverged.

To generate our own Lorenz attractor, we turn to numerical integration, specifically the Euler method due to its computational speed and efficiency. We use the classical Lorenz-63 parameter values, as this combination places the system in a

regime where the dynamics are chaotic but still bounded, producing the well-known butterfly attractor when plotted.

6. Analysis of reservoir dynamics on human connectome networks

6.1. Comparison with Gaussian reservoirs

To evaluate the performance of the connectome-based reservoir, we first compare it to a standard Gaussian random reservoir. Both reservoirs are trained to predict the next time step of the Lorenz system using identical training methods explained in previous sections, and their performance is compared using Root Mean Square Error (RMSE) over time on our generated Lorenz system. To create a similar comparison, we used identical parameters for both reservoirs, including the number of neurons ($N = 463$), the input scaling factor, and the spectral radius. The only difference is therefore in the internal structure of the reservoir.